

Git Going

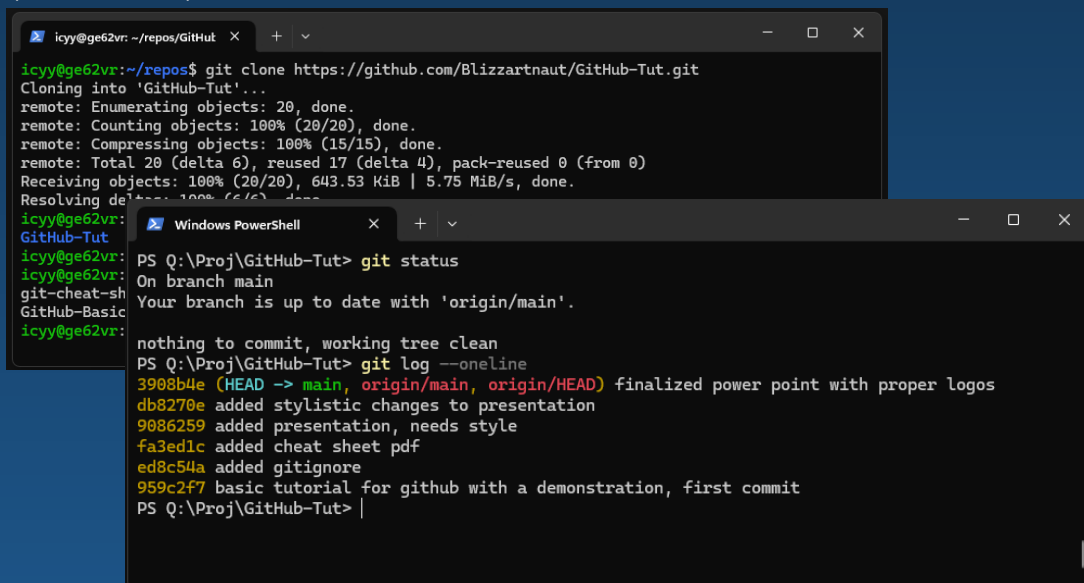
A Crash Course in Git and GitHub



Installing Git

- Command-line Client (CLI)
<https://git-scm.com/downloads>

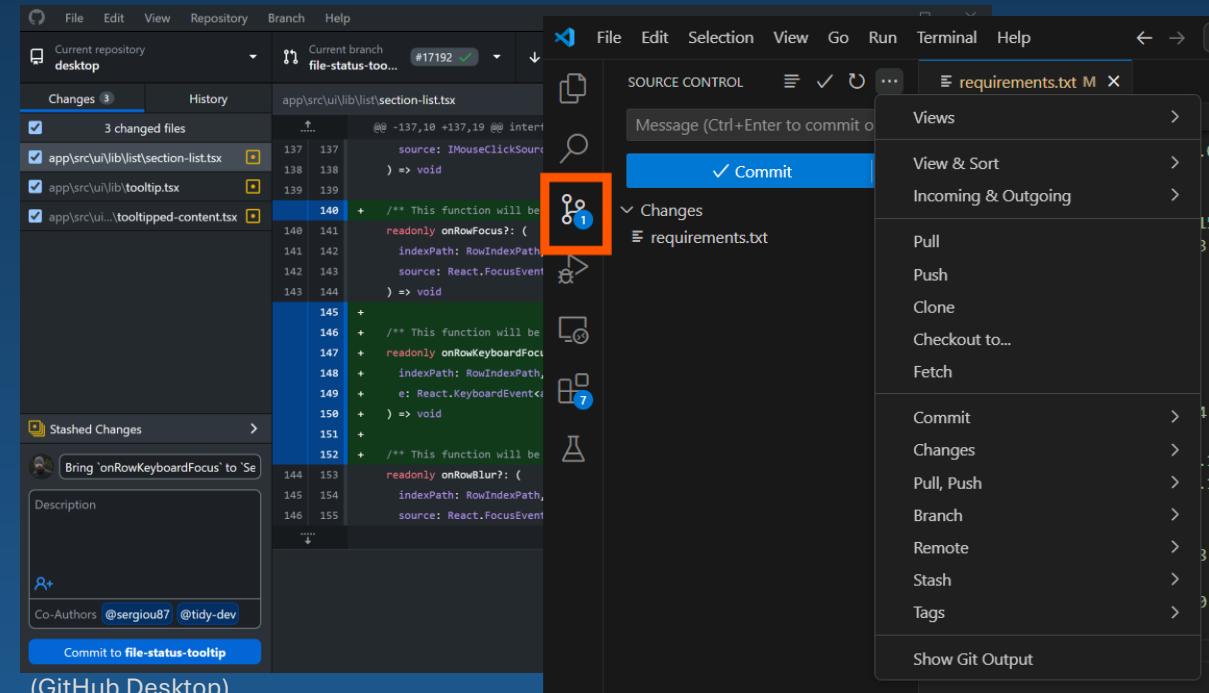
(Linux Terminal)



```
icyy@ge62vr: ~/repos/GitHub X
icyy@ge62vr:~/repos$ git clone https://github.com/Blizzartnaut/GitHub-Tut.git
Cloning into 'GitHub-Tut'...
remote: Enumerating objects: 20, done.
remote: Counting objects: 100% (20/20), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 20 (delta 6), reused 17 (delta 4), pack-reused 0 (from 0)
Receiving objects: 100% (20/20), 643.53 KiB | 5.75 MiB/s, done.
Resolving deltas: 100% (6/6), done.
icyy@ge62vr:
GitHub-Tut
icyy@ge62vr: PS Q:\Proj\GitHub-Tut> git status
On branch main
git-cheat-sh
GitHub-Basic
icyy@ge62vr:
nothing to commit, working tree clean
PS Q:\Proj\GitHub-Tut> git log --oneline
3908b4e (HEAD -> main, origin/main, origin/HEAD) finalized power point with proper logos
db8270e added stylistic changes to presentation
9086259 added presentation, needs style
fa3ed1c added cheat sheet pdf
ed8c54a added gitignore
959c2f7 basic tutorial for github with a demonstration, first commit
PS Q:\Proj\GitHub-Tut> |
```

(Windows PowerShell)

- Graphical Client (GUI)
<https://desktop.github.com/download>
Or use the one in your IDE!



(GitHub Desktop)

(VS Code)

Intro: Version Control and Git

- Version Control helps track changes you make in your code, and when collaborating with other programmers such as in a company environment allows you to keep track of everyone's changes.
- This allows for a mostly smooth experience of ensuring everyone is on the same page when programming.
- Its also important for engineers of all kinds for dealing with documentation, simulations, CAD files, or data analysis scripts.

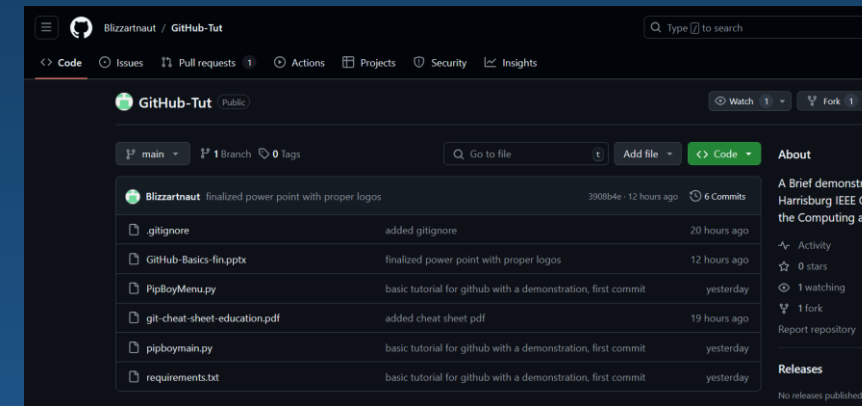
Git != GitHub

- **Git:** A distributed version control system, based on “snapshots” of your code
- Manages your source code
- Helps with collaboration



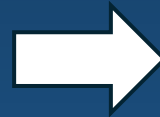
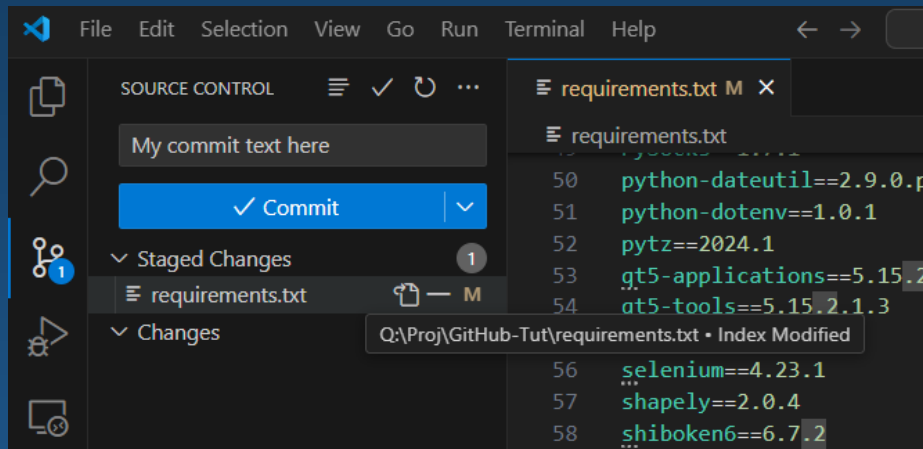
Linus Torvalds

- **GitHub:** A web-based hosting service for Git repositories allowing one to share their repositories with others.
- A Hub for Git projects. GitHub.
- Other platforms exist (GitLab, Gitea, etc.)

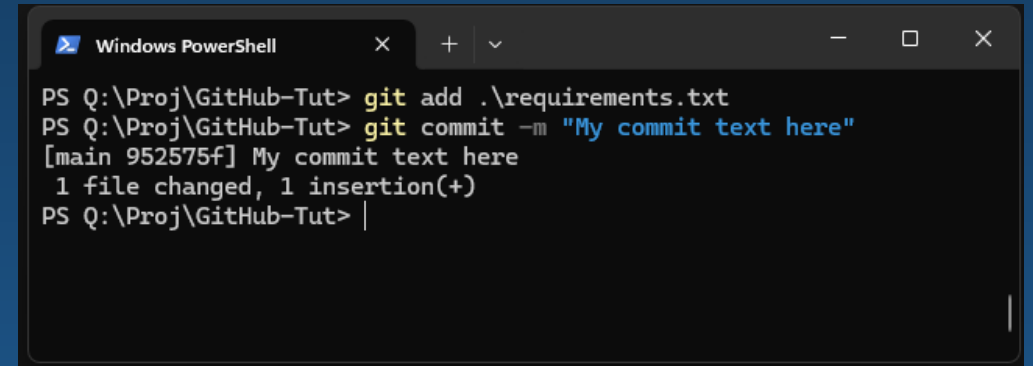


Git GUIs

- Convenient visual Git clients
- But they're just wrappers for the Git CLI

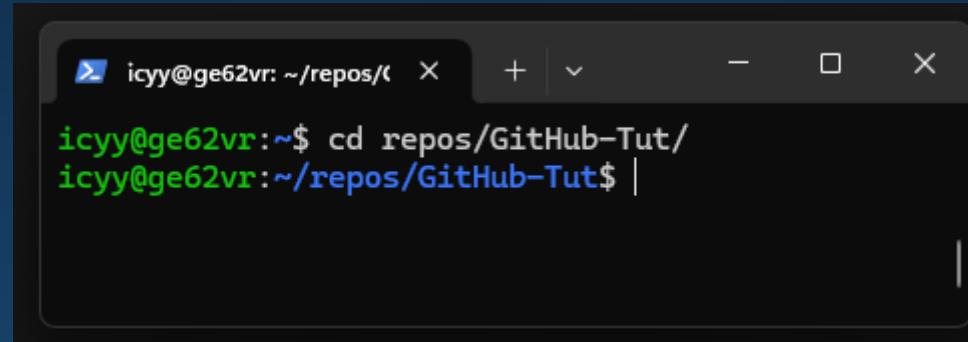


Same as



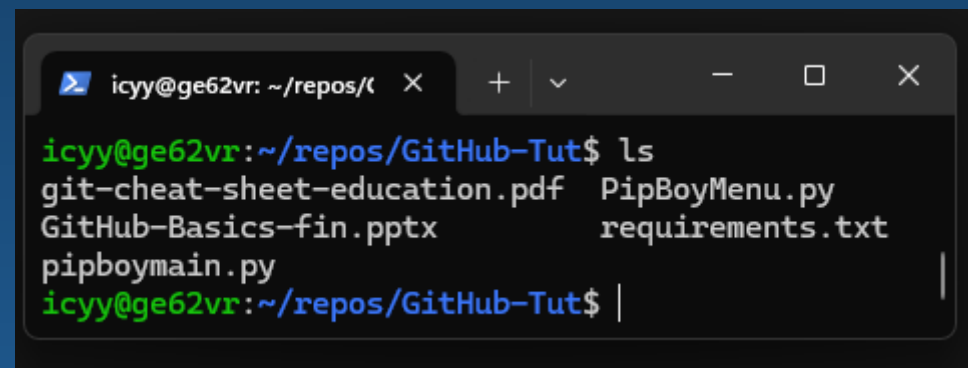
Quick Terminal Basics

- Change Directories with `cd`

A terminal window with a dark background. The title bar shows 'icyy@ge62vr: ~/repos/'. The prompt is 'icyy@ge62vr:~\$'. The user enters 'cd repos/GitHub-Tut/'. The prompt changes to 'icyy@ge62vr:~/repos/GitHub-Tut\$' with a cursor at the end.

```
icyy@ge62vr:~$ cd repos/GitHub-Tut/
icyy@ge62vr:~/repos/GitHub-Tut$ |
```

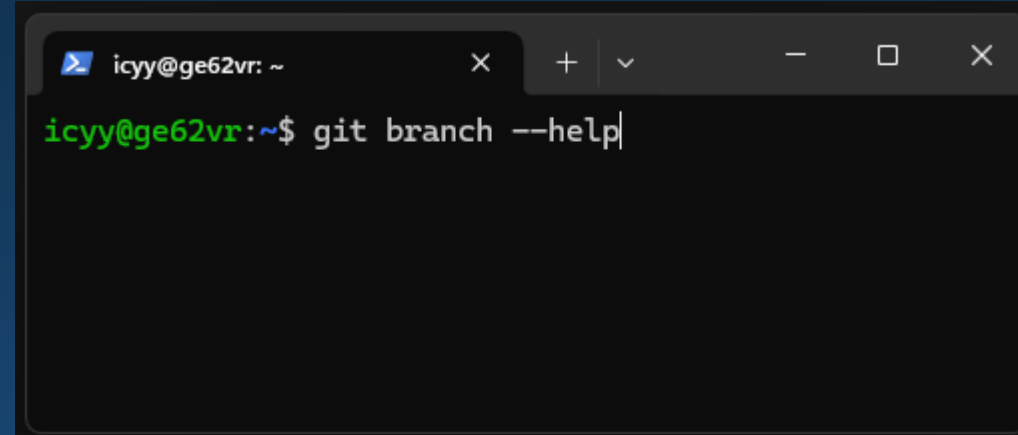
- List the files in your current directory with `ls`

A terminal window with a dark background. The title bar shows 'icyy@ge62vr: ~/repos/'. The prompt is 'icyy@ge62vr:~/repos/GitHub-Tut\$'. The user enters 'ls'. The output lists files: 'git-cheat-sheet-education.pdf', 'PipBoyMenu.py', 'GitHub-Basics-fin.pptx', 'requirements.txt', and 'pipboymain.py'. The prompt returns to 'icyy@ge62vr:~/repos/GitHub-Tut\$' with a cursor at the end.

```
icyy@ge62vr:~/repos/GitHub-Tut$ ls
git-cheat-sheet-education.pdf  PipBoyMenu.py
GitHub-Basics-fin.pptx        requirements.txt
pipboymain.py
icyy@ge62vr:~/repos/GitHub-Tut$ |
```

Quick Terminal Tips

- All commands have `--help`



```
icyy@ge62vr: ~  
icyy@ge62vr:~$ git branch --help
```

- You can also search for help online, or have chat AIs explain commands

Setting Up Git: Local Configuration

- After installation set up your local git basic settings
If you already have git you can ignore the steps on this page.
- Configure Username: (Type in terminal)

```
git config --global user.name "YourUsernameHere"
```
- Configure Email associated with your account (Type in terminal)

```
git config --global user.email "YourEmail@somewhere.com"
```

Your config gets saved to a file named `.gitconfig` in your user/home folder.



Git Basics: Definitions

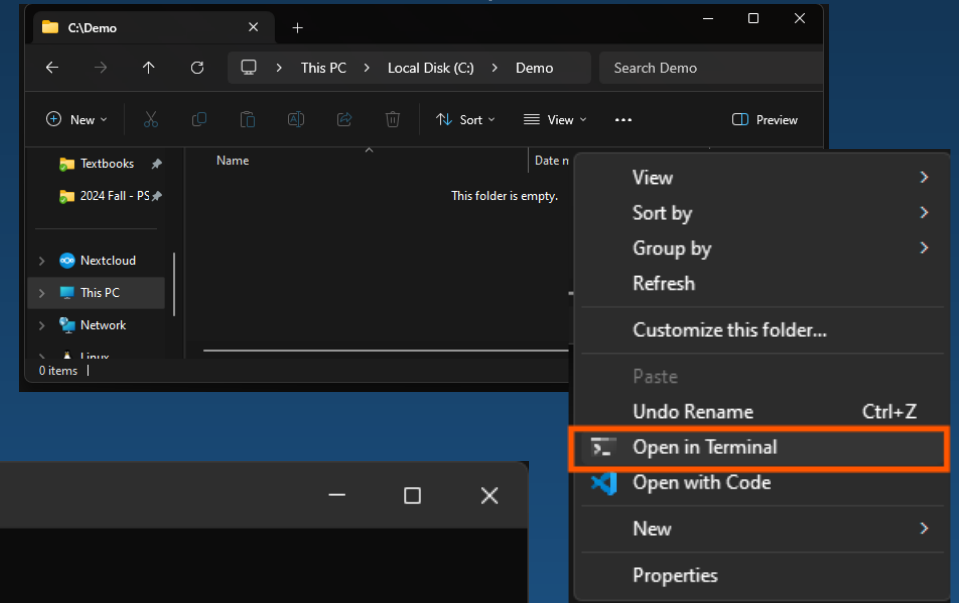
- Working Directory: The actual directory where project files are located, modifications made to files in the working directory are considered 'untracked' until staged for commit.
- Repository: "Repo", the database for all committed files, managing and tracking changes in files.



Creating a local Repo

- Navigate to a folder using your file explorer
- Or use just the terminal

with the file explorer



without the file explorer

A screenshot of a Windows PowerShell terminal window. The title bar says 'Windows PowerShell'. The text inside shows the copyright notice for Microsoft Corporation and a link to install the latest PowerShell. Below that, the command prompt shows the user 'sshpt' at 'C:\Users\sshpt>' typing the command 'cd C:\Demo'. The prompt then changes to 'PS C:\Demo>' and an orange arrow points to the end of the command line.

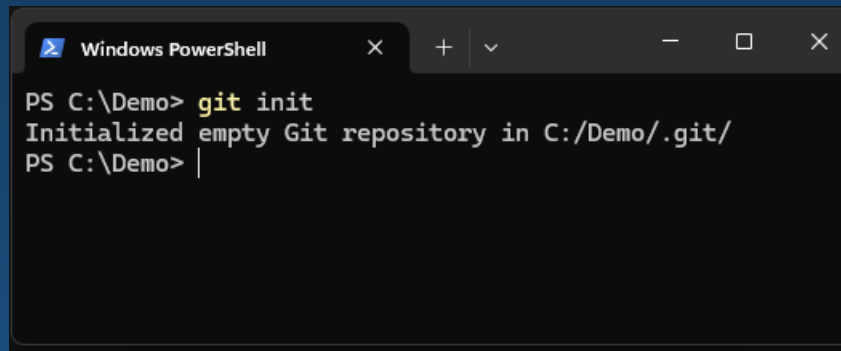
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\sshpt> cd C:\Demo
PS C:\Demo> |
```

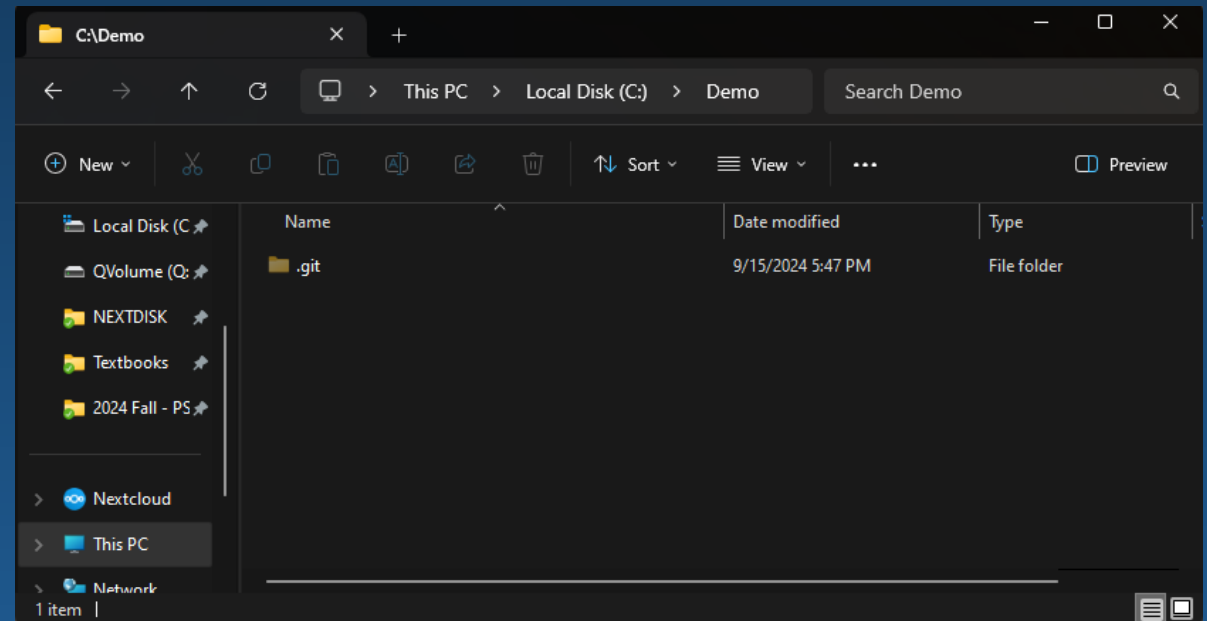
Creating a local Repo

- To create a new git repo, use `git init`
- This creates a hidden `.git` folder that contains Git's data.
- You can use this on blank or full folders. Anything can be a repository!



```
Windows PowerShell
PS C:\Demo> git init
Initialized empty Git repository in C:/Demo/.git/
PS C:\Demo> |
```

⚠ *Don't follow this yet!*
We're about to use git clone instead.

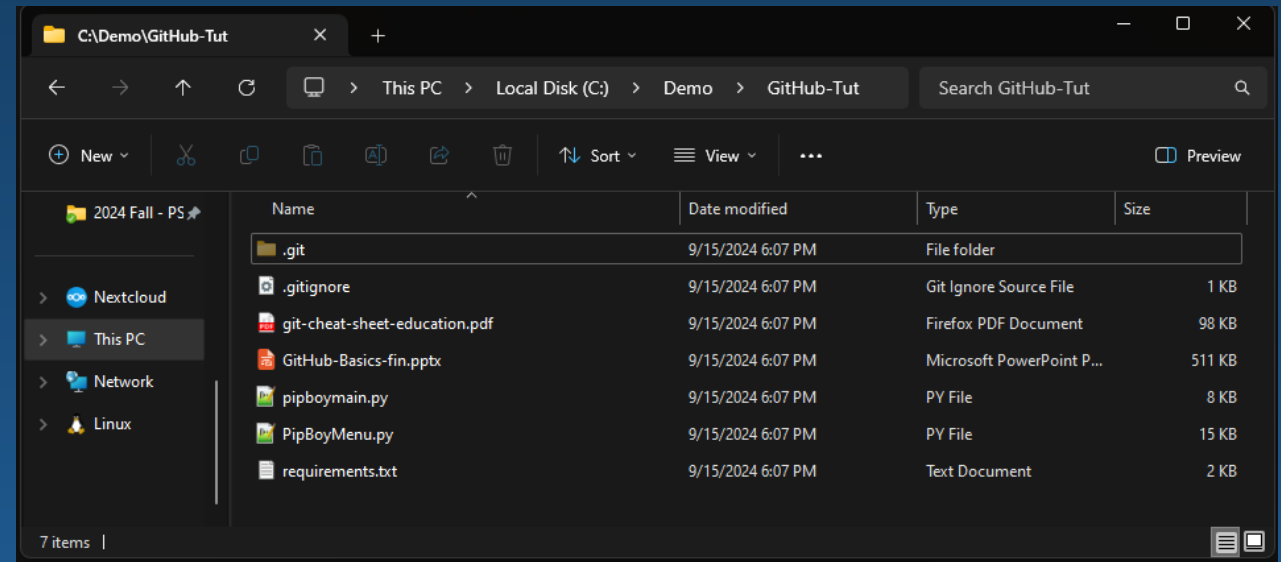


Cloning a Repo

- To clone an existing repo, use `git clone <url-to-repo>`
- This creates a new folder with the repository's latest contents
- As a demo, please run on your machine:

```
git clone https://github.com/Blizzartnaut/GitHub-Tut.git
```

```
Windows PowerShell
PS C:\Demo> git clone https://github.com/Blizzartnaut/GitHub-Tut.git
Cloning into 'GitHub-Tut'...
remote: Enumerating objects: 20, done.
remote: Counting objects: 100% (20/20), done.
remote: Compressing objects: 100% (15/15), done.
Receiving objects: 80% (16/20) 17 (delta 4), pack-reused 0 (from 0)
Receiving objects: 100% (20/20), 643.53 KiB | 5.27 MiB/s, done.
Resolving deltas: 100% (6/6), done.
PS C:\Demo> cd .\GitHub-Tut\
PS C:\Demo\GitHub-Tut>
```



Commits

- A commit is a “snapshot” of your source code
- A commit has:
 - A Snapshot (Of your files)
 - A Message (`git commit -m “Your Fancy Message”`)
 - An Author (Name & Email)
 - Parent commits (Can be multiple!)
- Each commit is identified by a SHA-1 Hash

Tracking Changes

- To check the difference between your working directory and the HEAD

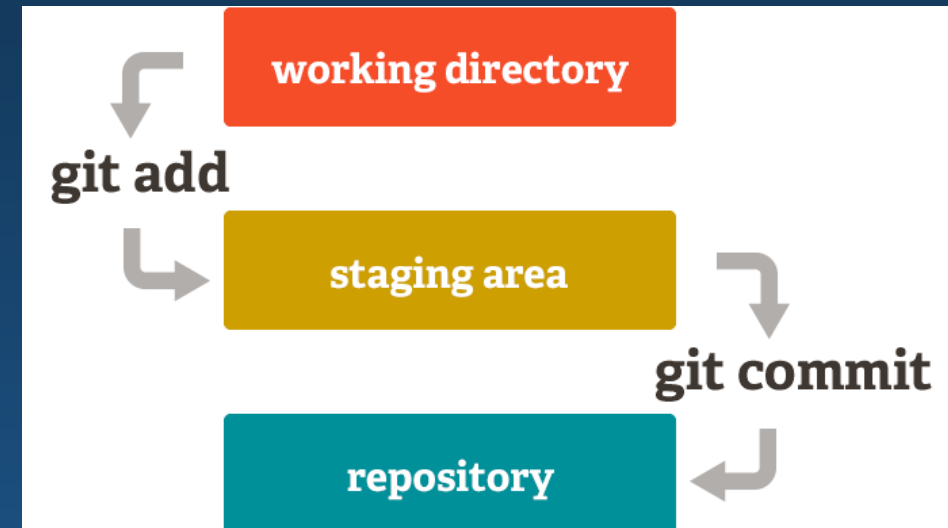
```
git status
```

- Add files to the staging area

```
git add file_name
```

- Create a new commit

```
git commit -m "commit message"
```



View commit history

- To display commit logs, use `git log`.
- Displays in *reverse chronological order* (Higher is newer)
- Press Q to quit the pager
- Very customizable! Useful flags:
 - `git log --oneline` . (Just the info you need)
 - `git log --graph` . (Shows branch merges really well)
 - `git log --all` . (Shows commits from other branches)
 - `git log --decorate` . (Shows ref names, enabled by default on newer versions)

View commit history

- To display commit logs, use `git log`.
- Displays in *reverse chronological order* (Higher is newer)
- Press Q to quit the pager
- Very customizable! Useful flags:
 - `git log --oneline` . (Just the info you need)
 - `git log --graph` . (Shows branch merges really well)
 - `git log --all` . (Shows commits from other branches)
 - `git log --decorate` . (Shows ref names, enabled by default on newer versions)
- You can combine flags!
 - `git log --all --decorate --oneline --graph` .
 - Use this quickly with aliases! To set this up, use:
`git config --global alias.logadog log --all --decorate --oneline --graph` .
And then you can just call `git logadog` . 🐕

References and Tags

- Using SHA-1 hashes to reference a commit isn't fun
- There's built in, human readable names that point toward specific commits
 - Branch names (e.g. **main**)
 - Tags, made by you or others (e.g. **v1.0.0**, **v6.11**)
 - **HEAD** - Your current branch's latest commit
 - **FETCH_HEAD** - The remote branch's latest commit
 - and more that don't belong in an intro to Git workshop...
- You can grab nearby commits using ~ and ^
 - **HEAD^** or **HEAD~1** – One commit before **HEAD**
 - Can get complicated, just read the friendly manual if you need something fancy ;)

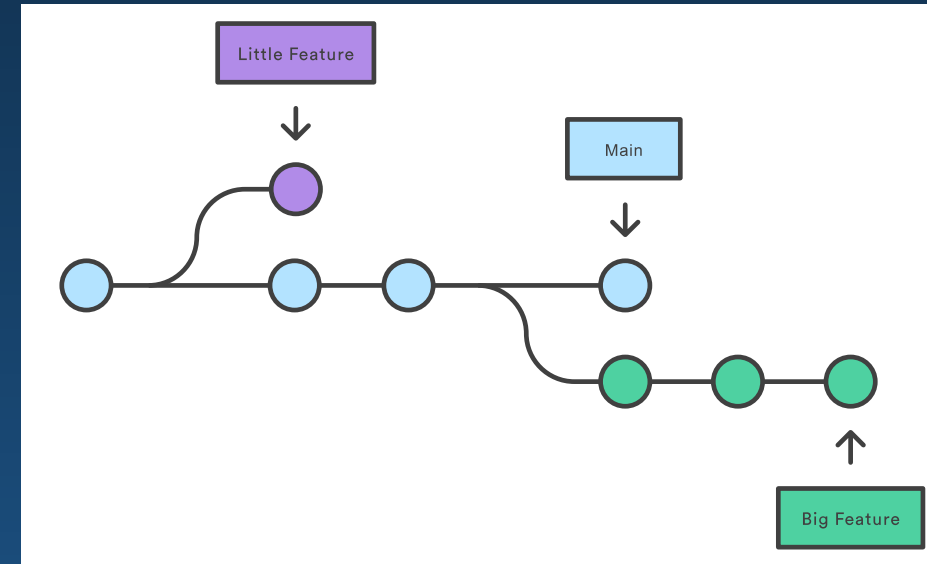


Differences between two commits

- To compare differences between commits or references
 - `git diff commitA..commitB` . (Difference between A and B)
 - `git diff commitA..HEAD` . (Difference between A and HEAD)
 - `git diff main..some-feature-branch` . (Difference between two branches)
 - `git diff HEAD^1..HEAD` . (Relative references work too!)
 - Mix and match!
 - Relative references work too!

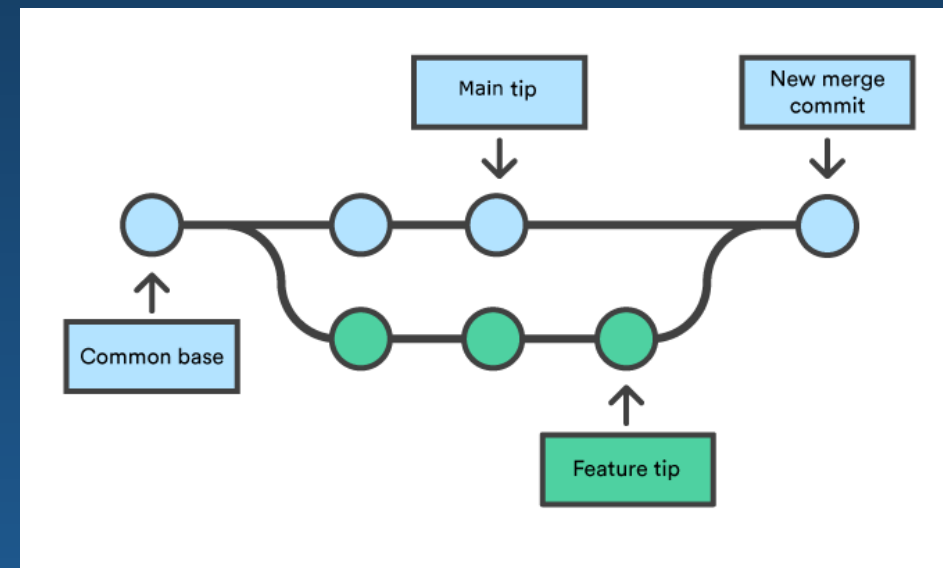
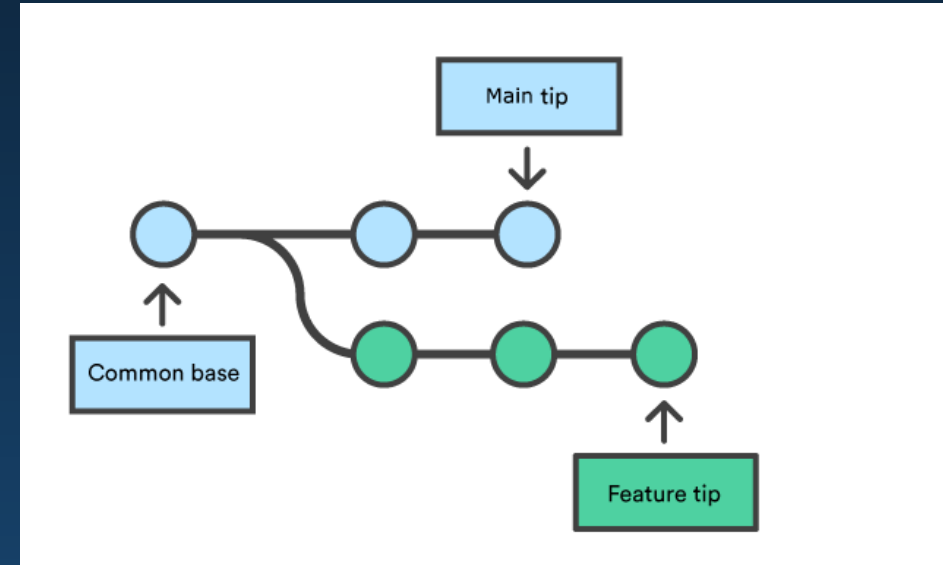
Branching Basics

- Branches are pointers to commits, and helps working in parallel
- Different projects have different naming/working standards
- Show existing branches
`git branch`
- Create a new branch named *new_feature*
`git branch new_feature`
- Switch to branch
`git checkout new_feature`
- Create AND switch to branch
`git checkout -b new_feature`



Git Merge

- Once you're done working on a branch, it's time to merge it. Git will handle wedging your changes into the existing codebase.
- Checkout the branch you want to merge on to `git checkout main`.
- Merge the branch `git merge new_feature`.
- Merged successfully? You can now delete your branch `git branch -d new_feature`.
- *Sometimes, merges don't go smoothly...*



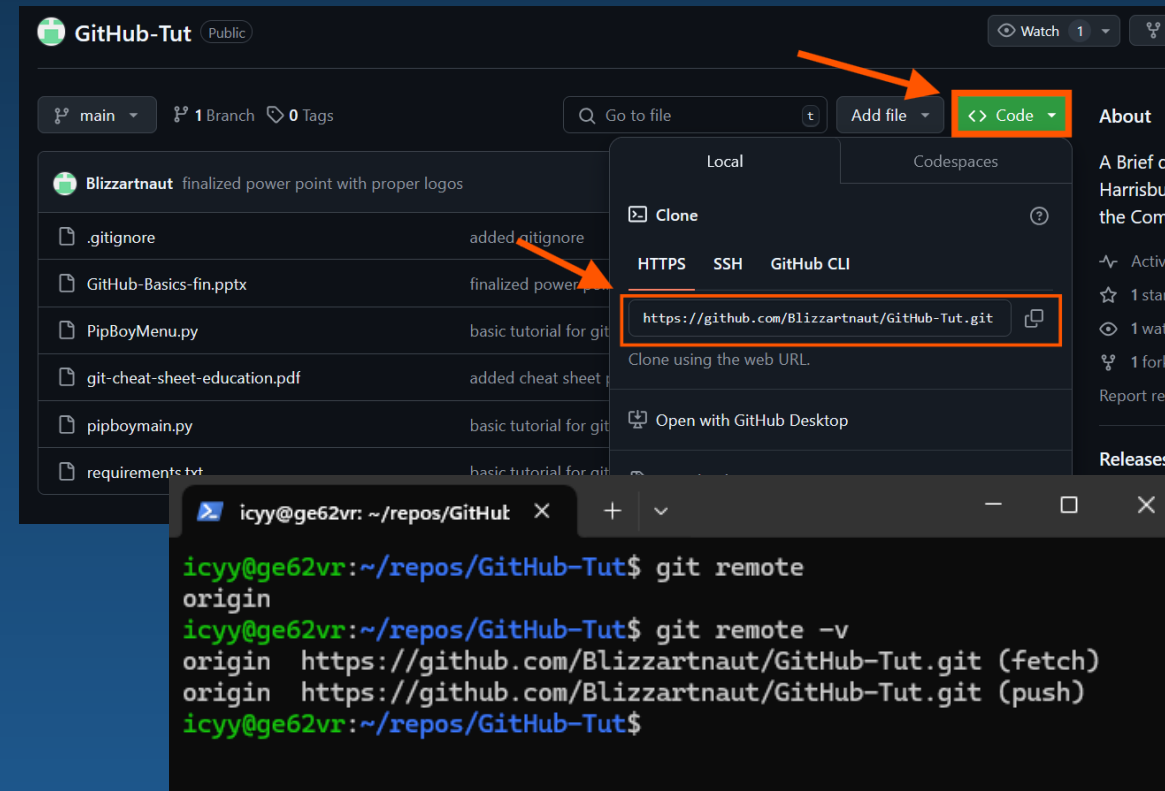
Merge Conflicts

- If Git can't cleanly merge, it will add markers to your file and let you know that there's a merge conflict.
- You will need to manually resolve the conflict to finish the merge
- The conflict is just text, you can either edit the file yourself or have your IDE help find and solve conflicts with one click
- `git add .` and `git commit`
Once you're done resolving.
Merge complete!

```
≡ sample.txt
1  These lines are either unchanged,
2  cleanly resolved because only one side changed,
3  or cleanly resolved because both sides changed the same way.
   Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
4  <<<<<< main:sample.txt (Current Change)
5  Text between these markers is from the branch you are on (current)
6  =====
7  Text between these markers is what is being merged (Incoming)
8  >>>>>> new_feature:sample.txt (Incoming Change)
9  And here is another line that is cleanly resolved or unmodified.
```

Remote Repositories

- Remotes offer a way to push the contents of your repository somewhere else, or pull the latest to your repository
- *Automatically* set if you use `git clone`
- `git remote` to view remotes
- `git remote -v` to see URLs
- `git remote add <remote_name> <URL>` to add another remote



Fetch, Pull, and Push

- As commits get pushed to the remote, you can use `git fetch` to tell Git to fetch the latest changes.
- Perform a pull to add the latest commits to one of your branches
`git pull <remote> <branch>` (example: `git pull origin main`)
 - Note: `git pull` also runs `git fetch` !
- Push your changes to the remote once you're ready
`git push <remote> <branch>` (example: `git push origin main`)
 - Note: Using `git push -u <remote> <branch>` lets you use `git pull` without any arguments!

Pipboy project Demonstration

- `git clone https://github.com/Blizzartnaut/GitHub-Tut`
- `python -m venv demo`
- `python demo/Scripts/activate`
- `pip install -r ./requirements.txt`
- `python PipBoyMenu.py`